Unité 8 — Commandes, chemins & XSS

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · 11 octobre 2026

> **Place dans le programme.** Suite de l'Unité 7 : trois autres injections applicatives parmi les plus courantes — **injection de commande système**, **traversée de chemin**, **XSS**. *Fondé sur meltypot (OS command injection `os.system` → `subprocess`, path traversal, XSS). Application de labo auteur, partagée avec l'Unité 7.*

1. Injection de commande système

Quand une entrée est concaténée dans une commande passée à un shell, l'attaquant enchaîne ses propres commandes (`;`, `|`, `&&`, `` `...` ``).

PYTHON — VULNÉRABLE (SHELL=TRUE + CONCATÉNATION)

```
@app.route("/ping")
def ping():
    host = request.args.get("host","127.0.0.1")
    out = subprocess.check_output("echo pong: " + host, shell=True)  # VULN : shell=True
    return out
```

Avec `host = 127.0.0.1;id`, le shell exécute `echo` *puis* `id`. Correctif : **ne pas invoquer de shell** (liste d'arguments) et **valider** l'entrée.

PYTHON — CORRECTIF

```
host = request.args.get("host","127.0.0.1")
if not all(c.isalnum() or c in ".-" for c in host):     # validation liste blanche
    return "hote invalide", 400
out = subprocess.check_output(["echo","pong:",host])     # liste d'args, pas de shell
```

2. Traversée de chemin (path traversal)

Quand un nom de fichier fourni par l'utilisateur est concaténé à un chemin de base sans confinement, `../` permet de sortir du dossier prévu.

PYTHON — VULNÉRABLE

```
name = request.args.get("name","notes.txt")
with open(os.path.join("/.../docs", name)) as f:         # VULN : pas de confinement
    return f.read()
```

Avec `name = ../../../../../etc/passwd`, on lit un fichier système. Correctif : résoudre le chemin réel (`realpath`) et vérifier qu'il reste **sous** le dossier de base ; idéalement, une **liste blanche** de noms.

PYTHON — CORRECTIF (CONFINEMENT)

```
full = os.path.realpath(os.path.join(base, name))
if not full.startswith(base + os.sep):                   # confinement
    return "acces refuse", 403
with open(full) as f: return f.read()
```

3. Cross-Site Scripting (XSS)

Quand une entrée est réinsérée dans une page HTML sans encodage, le navigateur de la victime exécute le script injecté.

PYTHON — VULNÉRABLE (RÉFLÉCHI)

```
return "<h1>Resultats pour %s</h1>" % q           # VULN : sortie non encodee
```

Avec `q = <script>alert(1)</script>`, le script est renvoyé tel quel et s'exécute. Trois types : **réfléchi** (dans la réponse immédiate), **stocké** (persistant en base), **DOM** (côté client). Correctif : **encodage de sortie** contextuel ; en défense, en-têtes `Content-Security-Policy`, cookies `HttpOnly`.

PYTHON — CORRECTIF (ENCODAGE)

```
return "<h1>Resultats pour %s</h1>" % escape(q)    # encodage de sortie
```

4. Le motif commun (rappel)

Les trois partagent la structure **source → sink** sans séparation code/données : shell, système de fichiers, HTML. La correction suit toujours la même logique : **séparer** (liste d'arguments, requête paramétrée), **confiner/valider** (liste blanche), **encoder** pour le contexte de destination.

5. Points clés

- Injection de commande : éviter `shell=True`, passer une liste d'arguments.
- Traversée de chemin : confiner sous un dossier de base, valider.
- XSS : encoder la sortie selon le contexte (HTML, attribut, JS).
- Même remède partout : séparer code et données, valider, encoder.

Exemples issus de meltypot ; application de labo auteur ; sorties produites et vérifiées.